



Лекція 15

**Веб-застосунок на NodeJs.
Робота з БД (MongoDB).
Тестування.**

Getting started

1. Install NodeJs
2. `npm i -g typescript`
3. Генеруємо та налаштовуємо **tsconfig.json** за допомогою команди **`npx tsc -init`**
4. Встановлюємо залежності.
Наприклад **`npm i express`** та **`npm i -D @types/express`** і т.д.
5. Налаштовуємо скрипти в **package.json**

```
"scripts": {  
  "start": "npm run build && node dist/app.js",  
  "build": "npx tsc"  
}
```
6. Готуємо структуру проекту та створюємо ініціалізаційний файл з налаштуванням фреймворка, підключення до бд, роутинг, логування та ін.

Dependencies

- **express** - це популярний веб-фреймворк для Node.js, який полегшує створення веб-додатків та API, надаючи безліч корисних функцій та зручний маршрутизатор.
- **mongoose** - це об'єктно-документна модель (ODM) для роботи з MongoDB у Node.js. Він надає простий та зручний спосіб визначення схем даних, виконання запитів та взаємодії з базою даних MongoDB
- **log4js** - це пакет для реєстрації логів, який забезпечує гнучку конфігурацію та безліч варіантів логування, включаючи різні цілі виведення та рівні логування.
- **class-transformer + class-validator** - трансформація даних у класи та їх валідація за допомогою декораторів.
- **dotenv** - пакет для завантаження змінних середовища з файлу .env у process.env, що дозволяє зберігати конфігурацію та секрети поза кодом (ключі, токени, налаштування середовища).

Development dependencies

- **tsx**- це інструмент для запуску TypeScript-коду напряму в Node.js з підтримкою watch-режиму, без попередньої компіляції в JavaScript.
- **eslint** - інструмент статичного аналізу коду, який перевіряє стиль, можливі помилки та порушення правил у JavaScript/TypeScript під час розробки, не виконуючи код.
- **prettier** - інструмент автоматичного форматування коду за єдиними правилами, без аналізу логіки чи помилок.

Test dependencies

- **jest** - фреймворк для запуску та організації тестів у JavaScript/TypeScript.
- **ts-jest** - адаптер для Jest, який дозволяє запускати TypeScript-тести без попередньої компіляції.
- **supertest** - бібліотека для тестування HTTP-серверів через запити до API без реального запуску мережі.
- **testcontainers** - інструмент для запуску залежних сервісів (наприклад, баз даних) у Docker-контейнерах під час тестування.

Структура проекта

- >  src
 -  .eslintrc
 -  .gitignore
 -  docker-compose.yml
 -  Dockerfile
 -  LICENSE
 -  package.json
 -  package-lock.json
 -  README.MD
 -  tsconfig.json

- ▼  src
 - >  config
 - ▼  controllers
 - >  groups
 - >  students
 -  ping.ts
 - >  dto
 - >  model
 - ▼  routers
 - >  groups
 - >  students
 -  index.ts
 - ▼  services
 - >  group
 - >  student
 - >  system
 - >  test
 -  app.ts
 -  index.ts

- ▼  src
 - >  config
 - >  groups
 - >  routers
 - ▼  students
 -  constants
 -  dto
 -  test
 -  types
 -  students.conrtoller.ts
 -  students.model.ts
 -  students.repository.ts
 -  students.router.ts
 -  students.service.ts
 -  app.ts
 -  index.ts

Dockerfile

Dockerfile описує процес збирання Docker-образу Node.js-застосунку: встановлення залежностей, збірку TypeScript та запуск готового застосунку.

```
FROM node:18-alpine as production-dependencies
WORKDIR /app
COPY package.json package.json
RUN npm install
```

```
FROM node:18-alpine as build-dependencies
WORKDIR /app
COPY package.json package.json
RUN npm install
```

```
FROM node:18-alpine as builder
WORKDIR /app
COPY --from=build-dependencies /app/node_modules node_modules
COPY . .
RUN npm run build
```

```
FROM node:18-alpine
WORKDIR /app
COPY --from=production-dependencies /app .
COPY --from=builder /app/build /app/build
CMD [ "node", "build/index.js" ]
```

- ✓ multi-stage build
- ✓ відділяємо build від runtime
- ✓ запускаємо скомпільований код

docker-compose.yml

docker-compose.yml - описує запуск застосунку разом із залежностями (MongoDB), налаштовує мережу, порти, змінні середовища та порядок старту сервісів.

```
version: '3.9'
services:
  nodejs-mongo-sample:
    build: .
    restart: on-failure
    ports:
      - '7777:7777'
    depends_on:
      mongo:
        condition: service_healthy
    networks:
      - default
    environment:
      - HOST=0.0.0.0
      - PORT=7777
      - MONGO_ADDRESS=mongodb://mongo:27017/mySampleDb
```

```
mongo:
  image: mongo:8.0.4
  ports:
    - '27019:27017'
  volumes:
    - ./docker/data/mongo/db:/data/db
  networks:
    - default
  healthcheck:
    test: ["CMD", "mongosh", "--eval",
"db.runCommand('ping').ok"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Just run this command in terminal:

`docker-compose up`

```
networks:
  default:
    driver: bridge
```


Application entry point

index.ts:

```
async function bootstrap(): Promise<void> {
  dotenvConfig();

  const { HOST, PORT, MONGO_ADDRESS } = useEnv();

  log4js.configure(config.log4js);

  const server = http.createServer(app);

  server.listen(PORT, HOST, async () => {
    log4js
      .getLogger()
      .info(`🚀 Example app listening on port: http://${HOST}:${PORT}`);
  });

  await connectToDatabase(MONGO_ADDRESS);
}

bootstrap();
```

Express app

1) app.ts:

```
const app = express();

app.use(cors());
app.use(express.json());
app.use('/', routers);
app.use(errorHandler);

export default app;
```

2) routers/index.ts:

```
const router = express.Router();

router.use('/api/groups', groups);
router.use('/api/students', students);

export default router;
```

3) student/student.router.ts:

```
const router = express.Router();

router.get('/:id', get);
router.post('', validateDto(StudentCreateDto), create);
router.patch('/:id', validateDto(StudentUpdateDto), update);
router.get('/groupId/:groupId', listByGroupId);
router.post('/_search', validateDto(StudentQueryDto), search);
router.delete('/:id', remove);

export default router;
```

Express params

```
router.get('/:id', get);
```

```
export const get = async (
  req: Request,
  res: Response,
  next: NextFunction,
): Promise<void> => {
  try {
    const id = req.params.id as string;
    const result = await studentService.get(id);
    if (!result) {
      res.status(StatusCode.NOT_FOUND).send();
    } else {
      res.send({
        ...result,
      });
    }
  } catch (error) {
    next(error);
  }
};
```

Express body

```
router.post('/', validateDto(StudentCreateDto), create);
```

```
export const create = async (
  req: Request,
  res: Response,
  next: NextFunction,
): Promise<void> => {
  try {
    const studentCreateDto = plainToInstance(StudentCreateDto, req.body);
    const id = await studentService.create(studentCreateDto);
    res.status(StatusCodes.CREATED).json({
      id,
    });
  } catch (error) {
    next(error);
  }
};
```

Express query

```
export const hello = async (
  req: Request,
  res: Response,
  next: NextFunction,
): Promise<void> => {
  try {
    const name = req.query.name;
    res.send(`Hello, ${name}!`);
  } catch (error) {
    next(error);
  }
};
```

Middleware

```
const app = express();  
app.use(errorHandler);
```

middleware/errorHandler.ts:

```
export function errorHandler(  
  err: any,  
  req: Request,  
  res: Response,  
  _next: NextFunction,  
): void {  
  if (err instanceof ValidationException) {  
    res.status(StatusCode.BAD_REQUEST).json({ errors: err.errors });  
  }  
  
  if (err instanceof NotFoundException) {  
    res.status(StatusCode.NOT_FOUND).json({ errors: err.message });  
  }  
  
  if (!res.headersSent) {  
    log4js.getLogger().error(err.stack);  
    res  
      .status(StatusCode.INTERNAL_SERVER_ERROR)  
      .json({ message: getReasonPhrase(StatusCode.INTERNAL_SERVER_ERROR) });  
  }  
}
```

class-transformer

- Перетворювати звичайні об'єкти (JSON) в екземпляри класів
- Перетворювати екземпляри класів назад у звичайні об'єкти (для JSON)

```
import { plainToInstance, instanceToPlain } from 'class-transformer';
```

```
class User {  
  id: number;  
  name: string;  
}
```

```
const json = { id: 1, name: 'Artem' };  
const user = plainToInstance(User, json);  
console.log(user instanceof User); // true
```

```
const plain = instanceToPlain(user);  
console.log(plain); // { id: 1, name: 'Bob' }
```

class-transformer

Дозволяє керувати серіалізацією та десеріалізацією через декоратори:

`@Expose()` — вказує, що поле буде включене при серіалізації.

`@Exclude()` — виключає поле.

`@Transform()` — дозволяє задати кастомну логіку перетворення.

```
import { Expose, Exclude, Transform } from 'class-transformer';

class User {
  @Expose()
  name: string;

  @Exclude()
  password: string;

  @Transform(({ value }) => value.toUpperCase())
  role: string;
}

const user = plainToInstance(User,
  { name: 'Bob', password: '123', role: 'admin' }
);
console.log(instanceToPlain(user));
// { name: 'Bob', role: 'ADMIN' }
```


class-validator

Дозволяє валідовувати дані класів через декоратори.

Зазвичай використовується разом із [class-transformer](#) і DTO

```
import { IsString, IsInt, Min, validate } from 'class-validator';

class UserDto {
  @IsString()
  name: string;

  @IsInt()
  @Min(18)
  age: number;
}

const user = new UserDto();
user.name = 'Bob';
user.age = 15;

validate(user).then(errors => {
  if (errors.length > 0) {
    console.log('Validation error:', errors);
  } else {
    console.log('Ok');
  }
});
```

class-validator

Популярні декоратори:

- `@IsString()` — поле має бути рядком
- `@IsInt()` — поле має бути цілим числом
- `@Min(value) / @Max(value)` — мінімум/максимум для чисел
- `@IsEmail()` — перевірка email
- `@Length(min, max)` — довжина рядка
- `@IsOptional()` — поле може бути пропущене
- `@IsDate()` — перевіряє, що поле є датою
- `@MaxDate(date)` — перевіряє, що дата не більша за вказану
- `@IsArray()` — перевіряє, що поле є масивом
- `@IsObject()` — поле має бути об'єктом
- `@IsNotEmptyObject()` — об'єкт не може бути пустим

Можна комбінувати кілька декораторів на одному полі.

class-validator + class-transformer

`@ValidateNested()` - Перевіряє вкладені об'єкти, тобто клас у класі. Часто використовується з `@Type()` з `class-transformer`. Дозволяє перевіряти **внутрішні класи**, а не тільки верхній об'єкт.

```
import { ValidateNested, IsString } from 'class-validator';
import { Type } from 'class-transformer';
```

```
class AddressDto {
  @IsString()
  street: string;
}
```

```
class UserDto {
  @IsString()
  name: string;

  @ValidateNested()
  @Type(() => AddressDto)
  address: AddressDto;
}
```

```
const user = new UserDto();
user.name = 'Bob';
user.address = new AddressDto(); // street не задано
validate(user).then(errors => console.log(errors));
```

Validation

```
router.post('', validateDto(GroupCreateDto), create);
```

```
const extractErrors = (errors: ValidationError[]): string[] => {  
  let messages: string[] = [];  
  errors.forEach((error) => {  
    if (error.children && error.children.length > 0) {  
      messages = messages.concat(extractErrors(error.children));  
    }  
    if (error.constraints) {  
      messages = messages.concat(Object.values(error.constraints));  
    }  
  });  
  return messages;  
};
```

```
export default (dtoClass: new () => object): RequestHandler => {  
  return async (req: Request, res: Response, next: NextFunction) => {  
    const dto = plainToInstance(dtoClass, req.body);  
    const errors: ValidationError[] = await validate(dto);  
    if (errors.length > 0) {  
      const messages = extractErrors(errors);  
      return next(new ValidationException(messages));  
    }  
    next();  
  };  
};
```

MongoDB

MongoDB – це популярна нереляційна система управління базами даних (NoSQL), яка використовує документно-орієнтовану модель (ODM) зберігання даних. Замість таблиць, як у реляційних базах даних, MongoDB зберігає дані у вигляді документів із довільною структурою, схожих на об'єкти JavaScript.

Запуск MongoDB в Docker

Файл *docker-compose.yml*

```
version: '3.9'
```

```
services:
```

```
  mongo:
```

```
    image: mongo:8.0.4
```

```
    container_name: 'mongo-sample-db'
```

```
    ports:
```

```
      - "27017:27017"
```

Запуск

```
docker-compose up
```

Mongo Console

mongosh

a6o

docker exec -it mongo-sample-db mongosh

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.2
PS C:\Users\Andrii> docker exec -it mongo-sample-db mongosh
Current Mongosh Log ID: 63eb5eca0e888c40a5101647
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.2
Using MongoDB:      5.0.14
Using Mongosh:       1.6.2

For mongosh info see: https://docs.mongodb.com/mongodbs-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2023-02-14T10:09:05.821+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http
://dochub.mongodb.org/core/prodnotes-filesystem
2023-02-14T10:09:06.382+00:00: Access control is not enabled for the database. Read and write access to data and configurati
on is unrestricted
2023-02-14T10:09:06.382+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'
-----

-----
Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make product
improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
-----

test> |
```

Приклад документу в форматі JSON

```
{
  "_id": ObjectId("6637194b003ec226154983f7"),
  "name": "Jhon",
  "surname": "Ho",
  "groupId": "6636af3000acc38e02d85256",
  "birthDate": ISODate("1995-12-11T22:00:00.000+0000"),
  "phoneNumbers": [
    "+3806688854533"
  ],
  "address": {
    "country": "country",
    "town": "town",
    "addressString": "addressString",
    "_id": ObjectId("6637194b003ec226154983f8")
  },
  "createdAt": ISODate("2024-05-05T05:29:47.657+0000"),
  "updatedAt": ISODate("2024-05-05T05:29:47.657+0000"),
  "__v": NumberInt(0)
}
```


Робота в консолі з MongoDB

```
// перелік всіх баз даних  
show databases;
```

```
// вибрати базу даних  
use <database_name>;
```

```
// подивитись всі колекції в вибраній базі даних  
show collections;
```

Робота в консолі з MongoDB

// вставка

```
db.students.insert({  
  "name": "Taras",  
  "surname": "Shevchenko",  
  "groupId": "6636af3000acc38e02d85256",  
  "birthDate": new ISODate('1814-03-09')  
});
```

// оновлення

```
db.students.update(  
  // критерій  
  { "name": "Taras" },  
  // встановлює значення полів  
  { $set: { "phoneNumbers": [ "+380667771235" ] } },  
  // оновлювати всі знайдені записи  
  { multi: true }  
);
```

Робота в консолі з MongoDB

// вибірка всіх записів колекції
`db.students.find();`

// вибірка 5-ти елементів, починаючи з 11-го
`db.students.find().skip(10).limit(5);`

// вибірка студентів з ім'ям і датою народження після
`db.students.find({
 "name": "Taras",
 "birthDate": { $gte: new ISODate('1800-01-01') }
});`

Підключення MongoDB в проєкті

```
import mongoose from 'mongoose';
```

```
await mongoose.connect(mongoUri);
```

MongoDB model

```
import mongoose, { Document, Schema } from 'mongoose';

export const MIN_LENGTH_GROUP_NAME = 2;
export const MIN_GROUP_START_YEAR = 2020;
export interface GroupData {
  _id: mongoose.Types.ObjectId;
  name: string;
  startYear: number;
}

const groupSchema = new Schema({
  name: {
    required: true,
    type: String,
    minlength: MIN_LENGTH_GROUP_NAME,
  },

  startYear: {
    required: true,
    type: Number,
    min: MIN_GROUP_START_YEAR,
  },
});

type GroupDocument = GroupData & Document;
const Group = mongoose.model<GroupDocument>('Group', groupSchema);
export default Group;
```

MongoDB model

```
await Group.find({});
```

```
const group = await new Group({  
  name,  
  startYear,  
}).save();
```

```
const students = await Student  
  .find({  
    ...(name && { name }),  
    ...(surname && { surname }),  
    ...(groupId && { groupId }),  
  })  
  .skip(query.skip)  
  .limit(query.limit);
```

```
await Student.findById(id);
```

```
await Student.findOneAndUpdate(  
  {  
    _id: id,  
  },  
  {  
    $set: {  
      ...studentDto,  
    },  
  },  
);
```

MongoDB model

Пошук

- `find()`
- `findOne()`
- `findById()`
- `exists()`
- `distinct()`

Створення

- `create()`
- `insertMany()`
- `new Model().save()`

Оновлення

- `updateOne()`
- `updateMany()`
- `replaceOne()`
- `findOneAndUpdate()`
- `findByIdAndUpdate()`

Видалення

- `deleteOne()`
- `deleteMany()`
- `findOneAndDelete()`
- `findByIdAndDelete()`

Підрахунок та агрегування

- `countDocuments()`
- `estimatedDocumentCount()`
- `aggregate()`

Додатково

- `populate()`
- `lean()`
- `select()`
- `sort()`
- `skip()`
- `limit()`

MongoDB model

```
const student = await Student.findById(id);
```

Що повертається:
екземпляр Mongoose Document

має методи:

- student.save()
- student.populate() - підвантажує пов'язані документи, замінює ObjectId на повноцінний документ
- student.toObject() - перетворює Document у plain JS object, прибирає внутрішній стан Mongoose

працюють:

- геттери / сеттери
- virtuals
- middleware
- є службові поля (_id, __v + внутрішній стан Mongoose)

Важче, повільніше, але “розумно”

MongoDB model

Геттеры / сеттеры:

```
const StudentSchema = new Schema({  
  name: {  
    type: String,  
    get: v => v.toUpperCase(),  
    set: v => v.trim(),  
  }  
});
```

```
student.name = '  bob  ';  
console.log(student.name); // 'BOB'
```

Virtuals:

```
StudentSchema.virtual('fullName').get(function () {  
  return `${this.firstName} ${this.lastName}`;  
});
```

MongoDB model

Middleware (hooks)

```
StudentSchema.pre('save', function () {  
  console.log('Before save');  
});
```

```
StudentSchema.post('save', function () {  
  console.log('After save');  
});
```

Document middleware

- save
- validate
- remove (deprecated)
- deleteOne
- init

Aggregate middleware

- aggregate

Query middleware

- find
- findOne
- updateOne
- countDocuments

и т.д.

Model middleware

- insertMany

MongoDB model

Метод `aggregate`

`aggregate()` — це pipeline обробки даних безпосередньо в MongoDB. Працюємо не з документом, а проганяємо дані через етапи.

```
await Student.aggregate([
  { $match: { groupId } },
  { $sort: { age: -1 } },
  { $limit: 10 },
]);
```

дані → фільтрація → трансформація → групування → результат
Кожен крок це { \$stage: { ... } }

Де `aggregate` справді потрібен

- + звіти
- + аналітика
- + статистика
- + складні вибірки
- + групування
- + dashboards

MongoDB model

Фільтрація (аналог find)

```
{ $match: { age: { $gte: 18 } } }
```

Вибір і трансформація полів

```
{ $project: { name: 1, age: 1, _id: 0 } }
```

Сортування / обмеження кількості / пропуск документів

```
{ $sort: { age: -1 } }  
{ $limit: 10 }  
{ $skip: 20 }
```

Групування + агрегації

```
{ $group: {  
  _id: "$groupId",  
  avgAge: { $avg: "$age" },  
  count: { $sum: 1 },  
} }  
[  
  { "_id": "A", "avgAge": 27.5, "count": 2 },  
  { "_id": "B", "avgAge": 20, "count": 1 }  
]
```



JOIN між колекціями

```
{ $lookup: {  
  from: "Group",  
  localField: "groupId",  
  foreignField: "_id",  
  as: "group",  
} }
```

Структура тестів

```
describe('Student Service', () => {  
  
    before(() => {  
    });  
  
    after(() => {  
    });  
  
    beforeEach(() => {  
    });  
  
    afterEach(() => {  
    });  
  
    it('createStudent should create a new student', () => {  
    });  
});
```

Assertions

`expect(value)` — точка перевірки, на якій викликаються методи порівняння.

Основні методи `expect`:

`toBe(value)` - Перевіряє суворе рівність (`===`)

`toEqual(value)` - Перевіряє глибоке рівність об'єктів/масивів

`toBeTruthy()` / `toBeFalsy()` - Перевіряє булеве значення

`toBeNull()` / `toBeUndefined()` / `toBeDefined()` - Перевірка на `null/undefined`

`toContain(item)` - Перевіряє, що масив або рядок містить елемент

`toHaveLength(n)` - Перевіряє довжину масиву або рядка

`toMatch(regex)` - Перевірка рядка на регулярний вираз

`toThrow()` - Перевірка, що функція викидає помилку

`toHaveBeenCalledWith(params)` - Перевірка, що функція була викликана з параметрами

Assertions

```
test('should throw NotFoundException', async () => {
  const studentId = new mongoose.Types.ObjectId().toString();
  jest.spyOn(studentRepository, 'get').mockResolvedValueOnce(null);
  await expect(studentService.get(studentId))
    .rejects.toThrow(NotFoundException);
  expect(studentRepository.get).toHaveBeenCalledWith(studentId);
});
```

```
test('should validate invalid groupId', async () => {
  await expect(
    studentRepository.create({
      name: 'John',
      surname: 'Doe',
      groupId: 'invalidId',
      birthDate: new Date('1990-01-30'),
    })
  ).rejects.toMatchObject({
    message: expect.stringContaining(
      'Cast to ObjectId failed for value "invalidId" (type string) at path "groupId"',
    ),
    name: 'ValidationError',
  });
});
```

```
const spy = jest.spyOn(service, 'method');
```

Моки результату:

- `mockReturnValue(value)` – синхронно повертає значення
- `mockReturnValueOnce(value)` – один раз
- `mockResolvedValue(value)` – `Promise.resolve(value)`
- `mockResolvedValueOnce(value)`
- `mockRejectedValue(error)` – `Promise.reject(error)`
- `mockRejectedValueOnce(error)`

Моки реалізації:

- `mockImplementation(fn)` – повністю замінює реалізацію
- `mockImplementationOnce(fn)` – один раз
- `mockName('name')` – ім'я для дебагу

```
expect(service.method).toHaveBeenCalledWith(someParams);
```

- `toHaveBeenCalled()`
- `toHaveBeenCalledTimes(n)`
- `toHaveBeenCalledWith(...args)`
- `toHaveBeenLastCalledWith(...args)`

MongoDB in tests

```
let container: StartedTestContainer | null = null;

export const startMongoContainer = async (): Promise<void> => {
  dotenvConfig({ path: `.env.test` });
  const { MONGO_PORT, MONGO_IMAGE, MONGO_DB } = useTestEnv();
  container = await new GenericContainer(MONGO_IMAGE)
    .withExposedPorts(MONGO_PORT)
    .start();
  const host = container.getHost();
  const port = container.getMappedPort(MONGO_PORT);
  const mongoUri = `mongodb://${host}:${port}/${MONGO_DB}`;
  await mongoose.connect(mongoUri);
};

export const stopMongoContainer = async (): Promise<void> => {
  if (mongoose.connection.readyState !== 0) {
    await mongoose.disconnect();
  }
  if (container) { await container.stop(); }
};

export const clearDatabase = async (): Promise<void> => {
  if (!mongoose.connection.db) {
    throw new Error('Database connection is not established');
  }
  await mongoose.connection.db.dropDatabase();
};
```

Тестуємо контролери

```
import request = require('supertest');  
import app from '../app';
```

```
const response = await  
  request(app).get(`/api/students/${id.toString()}`);  
expect(response.status).toBe(200);  
expect(response.body).toEqual(  
  expect.objectContaining({  
    name: studentFound.name,  
    surname: studentFound.surname,  
    groupId: studentFound.groupId,  
    birthDate: studentFound.birthDate!.toISOString(),  
  })),  
);
```

```
const app = express();
```

```
app.use(cors());  
app.use(express.json());  
app.use('/', routers);  
app.use(errorHandler);
```

```
export default app;
```

```
const response = await request(app)  
  .patch(`/api/students/${id.toString()}`)  
  .send(updateStudentDto);
```

Виклик зовнішніх API

```
const url = 'http://localhost:7777/api/someEndpoint';

const response = await fetch(
  url,
  {
    method: 'POST',
    headers: {
      'content-type': 'application/json;charset=UTF-8',
    },
    body: JSON.stringify(bodyToSend),
  }
);

const jsonResult = await response.json();
```

- **Axios**
- **SuperAgent**
- **Got**
- **Undici**



Дякуємо за увагу!

<https://github.com/ArtemPetrenkoEduardovich/nodejs-mongo-sample>